



Conditional Branching

Unlike Unconditional Branching Instructions, **Conditional Branching** instructions are only processed when a specific condition is met, based on the Destination and Source operands. A conditional jump instruction has multiple varieties as **Jcc**, where **cc** represents the **Condition Code**. The following are some of the main condition codes:

Instruction	Condition	Description
jz	D = 0	Destination equal to Zero
jnz	D != 0	Destination Not equal to Zero
js	D < 0	Destination is Negative
jns	D >= 0	Destination is Not Negative (i.e. 0 or positive)
jg	D > S	Destination Greater than Source
jge	D >= S	Destination Greater than or Equal Source

Instruction	Condition	Description
<code>jnl</code>	<code>D < S</code>	Destination <code>Less than</code> Source
<code>jle</code>	<code>D <= S</code>	Destination <code>Less than or Equal</code> Source

There are many other similar conditions that we can utilize as well. For a complete list of conditions, we can refer to the latest Intel [x86_64 manual](#), in the `Jcc-Jump if Condition Is Met` section. Conditional instructions are not restricted to `jmp` instructions only but are also used with other assembly instructions for conditional use as well, like the `CMOVcc` and `SETcc` instructions.

For example, if we wanted to perform a `mov rax, rbx` instruction, but only if the condition is `= 0`, then we can use the `CMOVcc` or `conditional mov` instruction, such as `cmovz rax, rbx` instruction. Similarly, if we wanted to move if the condition is `<`, then we can use the `cmovl rax, rbx` instruction, and so on for other conditions. The same applies to the `set` instruction, which sets the operand's byte to `1` if the condition is met or `0` otherwise. An example of this is `setz rax`.

RFLAGS Register

We have been talking about meeting certain conditions, but we have not yet discussed how these conditions are met or where they are stored. This is where we use the `RFLAGS` register, which we briefly mentioned in the Registers section.

The `RFLAGS` register consists of 64-bits like any other register. However, this register does not hold values but holds flag bits instead. Each bit 'or set of bits' turns to `1` or `0` depending on the

value of the last instruction.

Arithmetic instructions set the necessary 'RFLAG' bits depending on their outcome. For example, if a `dec` instruction resulted in a `0`, then bit `#6`, the Zero Flag `ZF`, turns to `1`. Likewise, whenever the bit `#6` is `0`, it means that the Zero Flag is off. Similarly, if a division instruction results in a float number, then the Carry Flag `CF` bit is turned on, or if a `sub` instruction resulted in a negative value, then the Sign Flag `SF` is turned on, and so on.

Note: When `ZF` is on (i.e. is `1`), it's referred to as Zero `ZR`, and when it's off (i.e. is `0`), it's referred to as Not Zero `NZ`. This naming may match the condition code used in the instructions, like `jnz` which jumps with `NZ`. But to avoid any confusion, let's simply focus on the flag name.

There are many flags within an assembly program, and each of them has its own bit(s) in the `RFLAGS` register. The following table shows the different flags in the `RFLAGS` register:

Bit(s)	0	1	2	3	4	5	6
Label (1 / 0)	<code>CF</code> (<code>CY</code> / <code>NC</code>)	<code>1</code>	<code>PF</code> (<code>PE</code> / <code>PO</code>)	<code>0</code>	<code>AF</code> (<code>AC</code> / <code>NA</code>)	<code>0</code>	<code>ZF</code> (<code>ZR</code> / <code>NR</code>)
Description	Carry Flag	Reserved	Parity Flag	Reserved	Auxiliary Carry Flag	Reserved	Zero Flag

Just like other registers, the 64-bit `RFLAGS` register has a 32-bit sub-register called `EFLAGS`, and a 16-bit sub-register called `FLAGS`, which holds the most significant flags we may encounter.

The flags we would mostly be interested in are:

- The Carry Flag **CF** : Indicates whether we have a float.
- The Parity Flag **PF** : Indicates whether a number is odd or even.
- The Zero Flag **ZF** : Indicates whether a number is zero.
- The Sign Flag **SF** : Indicates whether a register is negative.

All of the above flags are among the first few bits in the **FLAGS** sub-register. We will only be using the **jnz** instruction for our module project, which is applied whenever the **ZF** flag is equal to **0** (i.e., Not Zero **NZ**). So, let's see how to do so.

JNZ loopFib

The **loop loopFib** instruction we used in the last section is, in fact, a combination of two instructions: **dec rcx** and **jnz loopFib** , but since looping is a very common function, the **loop** instruction was created to reduce code size and be more efficient, instead of using both every time. However, the conditional jump instructions are much more versatile than **loop** , since they allow us to jump anywhere in the program on any condition we require.

Though it is more efficient to use **loop** , to demonstrate the use of **jnz** , let's go back to our code and try to use the **jnz** instruction instead of **loop** :

```
global _start

section .text
_start:
    xor rax, rax    ; initialize rax to 0
    xor rbx, rbx    ; initialize rbx to 0
```

nasm

```

    inc rbx          ; increment rbx to 1
    mov rcx, 10
loopFib:
    add rax, rbx     ; get the next number
    xchg rax, rbx    ; swap values
    dec rcx          ; decrement rcx counter
    jnz loopFib      ; jump to loopFib until rcx is 0

```

We see that we replaced `loop loopFib` with `dec rcx` and `jnz loopFib`, so that every time the loop reaches its end, the rcx counter would decrement by 1, and the program would jump back to `loopFib` if `ZF` is not set. Once `rcx` reaches `0`, the Zero Flag `ZF` would be turned on to `1`, and so `jnz` would no longer jump (since it's `NZ`), and we would exit the loop. Let's assemble our code, and run it with `gdb`, to see this in effect:

```

$ ./assembler.sh fib.s -g
gef> b loopFib
Breakpoint 1 at 0x40100e
gef> r

```

```

registers — $rax : 0x0
$rbx : 0x1
$rcx : 0xa
$eflags: [zero carry parity adjust sign trap INTERRUPT direction overflow resume virtualx86 identification]

```

```

registers — $rax : 0x1
$rbx : 0x1
$rcx : 0x9
$eflags: [zero carry PARITY adjust sign trap INTERRUPT direction overflow resume virtualx86 identification]

```

```

registers — $rax : 0x1
$rbx : 0x2

```

```
$rcx      : 0x8  
$eflags: [zero carry parity adjust sign trap INTERRUPT direction overflow resume  
virtualx86 identification]
```

We can see that we are still correctly calculating the Fibonacci Sequence. At each iteration of the loop, we are decreasing `rcx`, and the `zero` flag is off, while the `parity` flag is on when `rcx` is an odd number. The RFLAGS values at this point are set after the `dec rcx` instruction, as this is the last arithmetic instruction before we break. So, the flag states are for `rcx`.

Note: `GEF` shows us the state of the flags in the `RFLAGS` register. The flags written in bold UPPERCASE letters are on.

Let's continue out of the loop, to see the state of the registers and RFLAGS after `rcx` reaches `0`:

```
gef>  
Continuing.  
  
registers ——— $rax : 0x37  
$rbx      : 0x59  
$rcx      : 0x0  
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow RESUME  
virtualx86 identification]
```

We see that once `rcx` reaches `0`, the `zero` flag is set to on `1`, and `jnz` no longer jumps back to `loopFib`, so the program stops executing.

CMP

There are other cases where we may want to use a conditional jump instruction within our module project. For example, we may want to stop the program execution when the Fibonacci number is more than `10`. We can do so by using the `js loopFib` instruction, which would jump back to `loopFib` as long as the last arithmetic instruction resulted in a negative number.

In this case, we will not use the `jnz` instruction or the `rcx` register but will use `js` instead directly after calculating the current Fibonacci number. But how would we test if the current Fibonacci number (i.e., `rbx`) is less than `10`? This is where we come to the Compare instruction `cmp`.

The Compare instruction `cmp` simply compares the two operands, by subtracting the second operand from first operand (i.e. $D_1 - S_2$), and then sets the necessary flags in the `RFLAGS` register. For example, if we use `cmp rbx, 10`, then the compare instruction would do '`rbx - 10`', and set the flags based on the result.

Instruction	Description	Example
<code>cmp</code>	Sets <code>RFLAGS</code> by subtracting second operand from first operand (i.e. first - second)	<code>cmp rax, rbx -> rax - rbx</code>

So, after the first Fibonacci number is calculated, it will do '`1 - 10`', and the result would be `-9`, so it will jump since it's a negative number `<0`. Once we reach the first Fibonacci number greater than `10`, which is `13` or `0xd`, it will do '`13 - 10`', and the result would be `3`, at which case `js` would no longer jump, as the result is a positive number `>=0`.

We could use `sub` instructions to perform the same subtraction and set the flags if we wanted. However, this would not be efficient, as we will be changing the value of one of the registers, while

the `cmp` only compares and does not store the result anywhere. The main advantage of 'cmp' is that it does not affect the operands.

Note: In a `cmp` instruction, the first operand (i.e. the Destination) must be a register, while the other can be a register, a variable, or an immediate value.

So, let's change our code to use `cmp` and `js`, as follows:

```
nasm
global _start

section .text
_start:
    xor rax, rax    ; initialize rax to 0
    xor rbx, rbx    ; initialize rbx to 0
    inc rbx         ; increment rbx to 1
loopFib:
    add rax, rbx    ; get the next number
    xchg rax, rbx   ; swap values
    cmp rbx, 10     ; do rbx - 10
    js loopFib      ; jump if result is <0
```

Note that we removed the `mov rcx, 10` instruction since we are no longer looping with `rcx`. We could have used it in `cmp` instead of `10`, but by directly using `10` we use one less instruction, making our code shorter and more efficient.

Now, let's assemble our code, and run it with `gdb`, to see how this works. We will break at `loopFib`, and then step with `si` until we reach the `js loopFib` instruction:

```

$ ./assembler.sh fib.s -g
gef> b loopFib
Breakpoint 1 at 0x401009
gef> r

registers — $rax : 0x1
$rbx : 0x1
$eflags: [zero CARRY parity ADJUST SIGN trap INTERRUPT direction overflow resume
virtualx86 identification]

code:x86:64 —
0x401009 <loopFib+0> add rax, rbx
0x40100c <loopFib+3> xchg rbx, rax
0x40100e <loopFib+5> cmp rbx, 0xa
→ 0x401012 <loopFib+9> js 0x401009 <loopFib> TAKEN [Reason: S]

```

We see that in the first iteration of `loopFib`, once we reach `js loopFib`, the `SIGN` flag is set to on `1` as expected, since the result of `1 - 10` is a negative number. We also notice `GEF` telling us `TAKEN [Reason: S]`, which conveniently tells us that this conditional jump will be taken and gives the reason as `S`, meaning that the `SIGN` flag is set.

Now, let's `c`ontinue until `rbx` is greater than `10`, at which point `js` should no longer jump. Instead of manually pressing `c` several times, let's take this opportunity to learn how to set conditional breakpoints in `gdb`.

Let's first delete the current breakpoint with `del 1`, and then set our conditional breakpoint. The syntax is very similar to setting regular breakpoints `b loopFib`, but we add an `if` condition after it, such as `'b loopFib if $rbx > 10'`. Also, instead of breaking at `loopFib` and then using `si` to reach `js`, let's directly break at `js` with `*` to refer to its location `'b *loopFib+9 if $rbx > 10'` or `'b *0x401012 if $rbx > 10'`.

Remember: we can find an instruction's location with `disas loopFib`.

We see the following:

```
gef> del 1
gef> disas loopFib
Dump of assembler code for function loopFib:
..SNIP...
0x0000000000401012 <+9>:    js      0x401009
gef> b *loopFib+9 if $rbx > 10
Breakpoint 2 at 0x401012
gef> c

registers ——— $rax : 0x8
$rbx : 0xd
$eflags: [zero carry PARITY adjust sign trap INTERRUPT direction overflow resume
virtualx86 identification]

code:x86:64 ———
0x401009 <loopFib+0>    add     rax, rbx
0x40100c <loopFib+3>    xchg   rbx, rax
0x40100e <loopFib+5>    cmp     rbx, 0xa
→ 0x401012 <loopFib+9>    js      0x401009 <loopFib>    NOT taken [Reason: !(S)]
```

We see now that the last arithmetic instruction '13 - 10' resulted in a positive number, the **sign** flag is no longer set, so **GEF** tells us that this jump is **NOT TAKEN**, with the reason **!(S)**, meaning that the **sign** flag is not set.

As we can see, using conditional branching is very powerful and enables us to perform more advanced instructions based on a condition we specify. We can use the **cmp** instruction to test various conditions. For example, we can use **j1** instead of **jns**, which would jump as long as the Destination is Less than the Source. So, with **cmp rbx, 10**, **rbx** will start less than **10**, and once **rbx** gets greater than **10**, then **rbx** (i.e., the Destination) would be greater than **10**, at which point **j1** will not jump.

Note: We may see instructions using JMP Equal `je` , or JMP Not Equal `jne` . This is just an alias of `jz` and `jnz` , since if both operands are equal, the outcome of `cmp rax, rax` would be `0` in all cases, which sets the Zero Flag. The same applies to `jge` and `jnl` , since `>=` is the same as `!` , and applies to other similar conditions as well.

Now that we have covered all basic Control Instructions, which way do you think is more efficient?

1. Using `mov rcx, 10` and `loop loopFib` => loop 10 times
2. Using `mov rcx, 10` and `dec rcx` and `jnz loopFib` => jump 10 times
3. Using `cmp rbx, 10` and `js loopFib` => jump while `rbx < 10`

Modify your code to use the method you think is the best.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

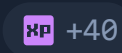
• Offline



Enable step-by-step solutions ⓘ PRO

⚠ Answer questions in English to ensure an accurate response.

Question 1



The attached assembly code loops forever. Try to modify (mov rax, 5) to make it not loop. What hex value prevents the loop?

Write your answer

 Submit

 Hint

 Download File

[← Previous](#)

Section 15 / 24

 +20

[Next](#)

[Mark Complete & Next](#)